

QUESTION 10(a)

Question:

To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>
#include <string.h>

int main()
{
    char current_dir[PATH_MAX];
    FILE *state_file;

    printf("==== Shell State and Exit ====\\n");

    /* Retrieve current directory */
    if (getcwd(current_dir, sizeof(current_dir)) == NULL)
    {
        perror("getcwd");
        return 1;
    }

    /* Display current path */
    printf("Current Directory: %s\\n", current_dir);

    /* Save state */
    state_file = fopen("shell_state.txt", "w");

    if (state_file == NULL)
    {
        perror("fopen");
        return 1;
    }

    fprintf(state_file, "Last Directory: %s\\n", current_dir);

    printf("Shell state saved successfully.\\n");
```

```
    /* Cleanup resource */
    fclose(state_file);

    printf("State file closed.\\n");

    /* Process exit request */
    char choice;

    printf("\\nDo you want to exit? (y/n): ");
    scanf(" %c", &choice);

    if (choice == 'y' || choice == 'Y')
    {
        printf("Cleaning up resources...\\n");
        printf("Exiting program safely.\\n");
        exit(0);
    }

    printf("Program continues running.\\n");

    return 0;
}
```

Output

```
Exiting shell...
sri_dhruthi@Dhruthi:~$ nano skill10a.c
sri_dhruthi@Dhruthi:~$ gcc skill10a.c -o skill10a
sri_dhruthi@Dhruthi:~$ ./skill10a
===== Shell State and Exit =====
Current Directory: /home/sri_dhruthi
Shell state saved successfully.
State file closed.

Do you want to exit? (y/n): y
Cleaning up resources...
Exiting program safely.
sri_dhruthi@Dhruthi:~$ cat shell_state.txt
Last Directory: /home/sri_dhruthi
sri_dhruthi@Dhruthi:~$ |
```

QUESTION 10(b)

Question:

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

```
GNU nano 8.7.1 skill10b.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX 200

int valid_name(char *name)
{
    if (name == NULL || name[0] == '\0')
        return 0;

    /* First character must be letter or underscore */
    if (!isalpha(name[0]) && name[0] != '_')
        return 0;

    /* Remaining characters */
    for (int i = 1; name[i] != '\0'; i++)
    {
        if (!isalnum(name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

int main()
{
    char input[MAX];
    char variable[MAX];
    char value[MAX];

    printf("===== Export Command Parser =====\n");

    printf("Enter export command:\n");
```

```

/* Check export keyword */
if (strncmp(input, "export ", 7) != 0)
{
    printf("Error: Invalid export syntax.\n");
    return 1;
}

char *assignment = input + 7;

/* Find '=' */
char *equal = strchr(assignment, '=');

if (equal == NULL)
{
    printf("Error: Missing '='.\n");
    return 1;
}

/* Extract variable name */
int name_length = equal - assignment;

if (name_length <= 0 ||
    name_length >= MAX)
{
    printf("Error: Invalid variable name.\n");
    return 1;
}

strcpy(variable, assignment, name_length);
variable[name_length] = '\0';

/* Extract value */
strcpy(value, equal + 1);

/* Validate variable name */
if (!valid_name(variable))

```

```

/* Display new value */
printf("New Value: %s\n", getenv(variable));

/* Create child process */
pid_t pid = fork();

if (pid < 0)
{
    perror("fork");
    return 1;
}

if (pid == 0)
{
    /* Child process */
    printf("\nChild Process:\n");
    printf("%s=%s\n", variable, getenv(variable));

    exit(0);
}
else
{
    /* Parent process */
    waitpid(pid, NULL, 0);

    printf("\nParent Process:\n");
    printf("Child process completed.\n");
}

return 0;

```

Output

```
sri_dhruthi@Dhruthi:~$ ./skill10b
===== Export Command Parser =====
Enter export command:
Example: export NAME=Dhruthi

$ export NAME=Dhruthi

Variable Name: NAME
Variable Value: Dhruthi
Existing Value: Dhruthi
Updating existing variable...
Environment variable updated successfully.
New Value: Dhruthi

Child Process:
NAME=Dhruthi

Parent Process:
Child process completed.
```